Notes on Combinatorial and Probabilistic Algorithms

Riccardo Lo Iacono

July 28, 2026

Contents.

– 1 – The analysis of Algorithms.

As a good computer scientist knows, not all algorithms are the same nor are the resouces these use; it is for these reasons that we need ways to analyze algorithms. In what follows, we first recall what asymptotic notation is, and then introduce the notion of *amortized analysis*.

– 1.1 – Asymptotic notation.

That idea behind asymptotic notation is that of analyzing algorithms, with respect to a given resource, more often then not being time, without caring about constants.

We recall the existence of three major notation: the *omega* notation (Ω), the *theta* notation (Θ) and the big-o notation ($\mathcal{O}$). Let $g(n)$ and $f(n)$ be two functions, then we define each notation as follow:

$$\begin{aligned}
\Omega(f(n)) =& \{g(n) \mid \exists c > 0, n_0 \geq 0 \text{ s.t. } g(n) \geq cf(n), \forall n \geq n_0\} \\
\Theta(f(n)) =& \{g(n) \mid \exists c_1, c_2 > 0, n_0 \geq 0 \text{ s.t. } c_1 f(n) \leq g(n) \leq c_2 f(n), \forall n \geq n_0\} \\
\mathcal{O}(f(n)) =& \{g(n) \mid \exists c, n_0 \geq 0 \text{ s.t. } g(n) \leq cf(n), \forall n \geq n_0\}
\end{aligned}$$

Although we give the definition in set notation, we write $g(n) = \mathcal{O}(f(n))$ to mean $g(n) \in \mathcal{O}(f(n))$; we do similarly for the other notations. Additionally, unless stated otherwise, assume we are interested in the time complexity of the algorithm.

– 1.2 – Amortized analysis.

Let us consider the following: assume we are given a data structure, and we are asked to compute a sequence of m operations on it; what is the overall complexity of all m operations, i.e, what is the time needed to compute all m operations.

It follows easily that, applying a worst case analysis, we need $m \cdot T_{MAX}(n)$, where $T_{MAX}(n)$ is the time complexity of the most expesive operation. For instance, under the worst case assumption, a sequence of m deletion from a tree has a complexity of $\mathcal{O}(m \cdot n)$.

We now introduce the notion of *amortized analysis*, and show how, over a sequence of operations, the real time complexity of an algorithm differs from the worst case one.

In its essence, amortized analysis does nothing more than averaging the complexity of all operations. That is, if $T_1(n)$ is the complexity of the first operation, $T_2(n)$ the one for the second operation, ..., $T_m(n)$ the complexity

of the m-th operation, then the amortized time is given as

$$T_{amortized}(n) = \frac{1}{n}\sum_{i=1}^{m} T_i(n).$$

In other words, amortized time represents the time needed for each of the m operations.

The next section describes a simple ways to perform amortized analysis.

– 1.2.1 – Accounting method.

The core idea of the method follows that used by banks. In more details: to each operation we assign a cost, which represents its *amortized cost*. When the amortized cost of an operation exeeds its real cost, we are left with some credit that we can use to help pay other operations.

Example: Consider we need an algorithm to increment a n bits counter by an amount m. From a worst case analysis perspective, the algorithm needs a quadratic time, as the n-th bit may need to be flipped m times. It comes with no surprise that such a cost is well above the real one, as we are about to see.

Let us proceed this way: for each bit flip, assume that we are asked to pay a coin; then we proceed by paying two coins when flipping a bit to 1. The cost of the whole algorithm is easily seen to be given by the number of total bit flipped. But the following can be observed: the first increment, leaves us with a coin of credit, the second one does the same, etc. Thus, each operation requires a constant amount; that is, the amortized cost of the whole algorithm is $\mathcal{O}(n)$.

Remark: Other methods to perform amortized analysis exist, and their use varies with the algorithm we need to analyze. Though, all follow the same core idea: we overpay for some operations and use the credit thus produced to pay for others.

– 2 – The dictionary problem.

Now, we focus our attention to one of the most common problems in computer science, namely, the *dictionary problem* which can be roughly defined as follow: given a set of items S, define a data structure such that the following three operations are performed efficiently.

Search(S, x)
Insert(S, x)
Delete(S, x)

The following assumption on operations cost is taken: accessing or deleting an item costs i while an insertion costs $i + 1$, where i is the length of the list preceeding the item.

We begin our discussion on the problem with the following trivial remark: if the elements have some kind of total order relation, then trees provide the best approach; if not, the best possible solution is given by lists.

Here we discuss two type of solutions, namely, self-adjusting lists (for the case in which no order relation exists) and self-adjusting trees (for the other case). Specifically, for self-adjusting trees we discuss, in order, red-black trees (Section 2.2.1) and splay trees (**??**).

– 2.1 – Self-adjusting lists.

Our discussion on self-adjusting lists will proceed as follow: first, we introduce the definition of self-adjusting list; then, we focus our attention on some of the most common rules used to keep a list organized.

Conceptually, self-adjusting lists are trivial: these are lists in which, after accessing or inserting an item, some update procedure is called to keep the list organized. These procedure are defined according to some rules, among which those that stands out are

- *Frequency Count (FC)*: we mantain an array that stores at each position i, the count of how many times item i has been accessed or added;
- *Move-To-Front (MTF):* each time an item is added, it is moved to the head of the list, while each accessed item is moved slighly closer to the root;
- *Transpose (T):* accessing or inserting an item, implies an exchange of the added/accessed item with the one on its left.

Let $\mathbb{E}_A(p)$ be the expected cost to access an item using algorithm A, where $p = \{p_1, p_2, \ldots, p_n\}$ is the probability distribution associated to the sequence of operations. Assume DP to be an algorithm that orders the elements according

to p. By the law of large numbers it follows $\mathbb{E}_{FC}(p)/\mathbb{E}_{DP}(p) = 1$. Additionally, one could prove that $\mathbb{E}_{MTF}(p) \leq 2\mathbb{E}_{DP}(p)$, and as proved in [1] $\mathbb{E}_T(p) \leq \mathbb{E}_{MTF}(p)$. A similar result can be easily derived for insertions and deletions. From the above, it follows that, at least in theory, both FC and transpose out perform MTF. As we are about to show, theory and practice not always coincide. What we want to show is that, although in theory FC and transpose are better then MTF, in practical applications MTF performs better.

For any algorithm A, let us denote with $C_A(s)$ the total cost of all operations, by $F_A(s)$ the number of exchanges that move an item to the head of the list, and by $P_A(s)$ any other exchange. The following holds.

Theorem 2.1 ([2, Theorem 1.]): *For any Algorithm A and any sequence of operations s starting with the empty set*

$$C_{MTF}(s) \leq C_A(s) + P_A(s) - F_A(s) - m.$$

Theorem 2.1 and its generalized version (see [2, Theorem 2.]), show that MTF differs by any other algorithm by a most a factor of 2. No analogous result holds for FC and transpose.

– 2.2 – Self-adjusting trees.

Our discussion on self-adjusting trees will proceed as follow: first, we introduce Red-Black Trees (RB) and we overview the operations on these, then, we describe splay trees.

– 2.2.1 – Red-Black trees.

A special variant of Binary Search Trees (BST), RB trees allow to implement the operations of Search(S, x), Insert(T, x) and Delete(T, x) in $\mathcal{O}(\log n)$ time.

Formally speaking, RB trees are binary search trees satisfying the following properties:

1. Each internal node is either red or black.
2. The root is black.
3. Each leaf is black.
4. Each red node has only black children.
5. Any simple path from a node to a descendant leaf, contains the same number of black nodes.

Property 5 implicitly defines a measure to which, for a given node x, we refer to as the black height $\texttt{bh}(x)$. That is, $\texttt{bh}(x)$ is the number of black nodes

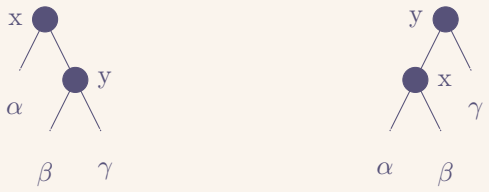
in the path from x to any of its descendant leaf. The following result easily follows.

Lemma 2.1: *If T is a Red-Black tree with n nodes, then T has height at most* $2\log(n+1)$.

Proof. Follows immediately by a simple induction on n. ■

Rotations.

As we shall see, insertion and deletion on a RB tree are everything but simple. We shall in fact see that either of the two operations may cause one or more properties to fail. To fix this issues specific procedures are used to restore each property; in doing so some local pointer manipulations are made, which here we refer to as rotations. We describe left rotations, the other being symmetrical.



(a) **(b)**

Figure 1: An example of left (resp. right) rotation.

Consider the case shown in Figure 1.a. To obtain the tree in Figure 1.b we proceed as follow: we let y's left child to be x's new right child, and we make x be the new y's left child.

Insertion.

In our discussion on RB insertion we assume the following: for any node x, $x.r$ denotes the right child of x, likewise $x.l$ denoets the left child, $x.p$ denotes x's parent, and $x.c$ is x's color. Additionally, for any inserted node x, $x.l$ and $x.r$ are set to nil, which we assume to be black, and $x.c$ is red. We refer to the inserted node with z.

We now discuss RB insertion focusing on which of the RB properties may fail after insertion, and how these are fixed.

First, observe that after insertion every node is either red or black, and thus Property 1 holds, similarly Property 3 holds as the newly inserted node has $z.l$ and $z.r$ set to nil, and so does Property 5 since no new black node has been added. On the other hand, Property 2 may fail if the inserted node replaces the root, while Property 4 fails when $z.p$ is red.

With respect to Procedure RB-Insert, which shows the pseudo code to implement RB insertion, we show how RB−Insert−Fixup restores each of the RB

Procedure RB-Insert(T, z)

```
y = nil ;
x = T.root
while x ≠ nil do
    y = x
    if z.key < x.key then
        x = x.left
    else
        x = x.right
end
z.p = y
if y == nil then
    T.root = z
else if z.key < y.key then
    y.left = z
else
    y.right = z
z.left = nil
z.right = nil
z.color = "Red"
RB-Insert-Fixup(T, z)
```

properties.

Case 1: z's uncle y is red. Since both $z.p$ and y are red, and $z.p.p$ exists and is black, we simply "push" the redness upward and call RB−Insert−Fixup on $z.p.p$. That is, we recolor $z.p$ and y black, while recoloring $z.p.p$ red.

Case 2: z's uncle y is black and z is a right child. To be considered a subcase of Case 3, we simply call Left−Rotate on z and fall into Case 3.

Case 3: z's uncle y is black and z is a left child . Either it is accessed directly or through Case 2, we simply call Right−Rotate on z and apply some recoloring.

The only property that may still fail is Property 2, which a simple recoloring at the end of RB−Insert−Fixup solves.

It shall be evident that upon RB−Insert−Fixup termination, all RB properties are restored.

Procedure RB-Insert-Fixup(T, z)

```
while z.p.color == "Red" do
    if z.p == z.p.p.left then
        y = z.p.p.right ;
        if y.color == "Red" then
            z.p.color = "Black"
            y.color = "Black"
            z.p.p.color = "Red"
            z = z.p.p
        else if z == z.p.right then
                z = z.p
                Rotate-left(T, z)
            z.p.color = "Black"
            z.p.p.color = "Red"
            Rotate-right(T, z.p.p)
    else
        /* Likewise with "right" and "left" swapped.       */
end
T.root.color = "Black"
```

Deletion.

Our discussion on RB deletion will assume the following: for any node y, $x.r$ denotes x's right child, similarly $x.l$ denotes the left child, with $x.p$ we indicate

x's parent and with $x.c, x$'s color. Additionally, we denote with z the node to remove, with y the node the replaces the deleted node and we let it always point to z, and with x the node filling y's position after this has been moved.

As for insertion, we mainly focus on the properties that may fail upon deletion, and explain how these are fixed.

First, if y was the root and a red child replaces it, Property 2 fails. Second, if x and $x.p$ are both red, then Property 4 fails. Third, moving y may cause Property 5 to fail. We can correct the violation of Property 5 by saying that node x, now occupying y's original position, has an "extra" black. That is, if we add 1 to the count of black nodes on any simple path that contains x, then under this interpretation, property 5 holds. When we remove or move the black node y, we "push" its blackness onto node x. The problem is that now node x is neither red nor black, thereby violating property 1.

Procedure RB-delete(T, z)

```
y = z
y − starting − color = y.color
if z.left == nil then
    x = z.right
    RB − transplant(T, z, z.right)
else if z.right == nil then
    x = z.left
    RB − transplant(T, z, z.left)
else
    y = Tree − Min(z.right)
    x = y.right
    if y.p == z then
        x.p = z
    else
        RB − transplant(T, y, y.right)
        y.right = z.right
        y.right.p = y
    RB − transplant(T, z, y)
    y.left = z.left
    y.left.p = y
    y.color = z.color
if y − starting − color == "Black" then
    RB − delete − fixup(T, x)
```

With respect to Procedure RB-delete, which shows the pseudo code to implement RB deletion, we show how RB−Delete−Fixup restores each of the RB

properties.

Case 1: z's sibling w is red. We know that w's must be black, therefore we can exchange the colors of w and $x.p$ and apply a left rotation on $x.p$.

Case 2: z's sibling w is black, and so are its children. Since both w's children are black, and so are z and w, we can simply pulling away a black from both, leaving z singly black and w red.

Case 3: z's sibling w is black, w.l is red and w.r is black. We can switch the colors of w and its left child $w.l$ and then perform a right rotation on w.

Case 4: z's sibling w is black, and w.r is red. By making some color changes and performing a left rotation on $x.p$, we can remove the extra black on x, making it singly black.

Upon termination Property 2 may still fail, which is fixed at the end of RB–Delete–Fixup.

Procedure RB-delete-fixup(T, x)

```
while x ≠ T.root and x.color ≠ "Black" do
    if x == x.p.left then
        w = x.p.right
        if w.color == "Red" then
            w.color = "Black"
            x.p.color = "Red"
            Rotate − left(T, x.p)
        if w.left.color == "Black" and w.right.color == "Black"
         then
            w.color = "Red"
            x = x.p
        else if w.right.color == "Red" then
            w.left.color = "Black"
            w.color = "Red"
            Rotate − right(T, w)
            w = x.p.left
        w.color = x.p.color
        x.p.color = "Black"
        Rotate − left(T, x.p)
        x = T.root
    else
        ( same as above with "right" and "left" exchanged )
end
x.color = "Black"
```

References.

[1] R. L. Rivest. "On Self-Organizing Sequential Search Heuristics". In: *Commun. ACM* 19.2 (1976), pp. 63–67. DOI: 10.1145/359997.360000.

[2] D. D. Sleator and R. E. Tarjan. "Amortized Efficiency of List Update and Paging Rules". In: *Commun. ACM* 28.2 (1985), pp. 202–208. DOI: 10.1145/2786.2793.